PIMPRI CHINCHWAD EDUCATION TRUST'S

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

SECTOR NO. 26, PRADHIKARAN, NIGDI, PUNE - 411044

MDM CAPSTONE PROJECT SYNOPSIS

Project Group ID: Grp_Cap_14

“AI-Driven Community Intelligence and Knowledge Discovery Platform ”

Project Group Members:

PRN No.	Name of Students
123B1B003	Aditya Atre
123B1B004	Aaryan Bhujang
123B1B014	Pushkar Kirange
123B1B021	Ishan Patil

Dr. Sonal Gore

Project Guide **Head Of Department**

DEPARTMENT OF COMPUTER ENGINEERING

ACADEMIC YEAR 2026 - 2027

1. INDEX

Sr. No.		Contents	Page No.
1		Index	1
2		Abstract	2
3		Introduction	3
4		Background	4
	4.1	Problem Statement	4
	4.2	Motivation / Need of Project	4
	4.3	Objectives	4
	4.4	Scope of the Project	5
5		Literature Survey	6
6		Methodology	9
	6.1	Block Diagram	9
	6.2	Hardware and Software Requirement	10
	6.3	Flowchart	11
	6.4	Working Principle	13
7		Expected Conclusions	15
8		References	16

2. ABSTRACT

The Industry Skill Development and Innovation Cell (ISDIC) operates campus-embedded skill development centres and city-level community chapters across four technology domains: cybersecurity, artificial intelligence and machine learning, cloud and DevOps, and software development. As the organisation expands across institutions and cities, information describing its members, chapters, curriculum tracks, events, tools and industry partners accumulates across structured records and unstructured documents that cannot presently be queried through a single interface. ISDIC has accordingly identified the requirement of an intelligent chatbot capable of answering questions over its existing community data.

This project proposes a Community Intelligence Platform in which such a chatbot forms the core module, built upon a hybrid retrieval architecture that combines a Knowledge Graph with vector-based Retrieval-Augmented Generation (RAG). Community records and documents are processed into entities and relationships stored in a knowledge graph, while textual content is embedded into a vector database. A hybrid retrieval layer resolves both factual and relationship-oriented queries and supplies grounded context to a Large Language Model (LLM), which generates the final response. Three supporting modules extend the same architecture without duplicating it: natural-language community search with structured filtering, explainable recommendation of members, events and opportunities, and generative community insights for administrators.

The expected outcome is a single, grounded and verifiable interface to community knowledge that reduces manual coordination effort, surfaces relationships that are presently invisible in tabular records, and demonstrates the practical application of Generative AI to community intelligence.

Keywords: Generative AI, Retrieval-Augmented Generation, Knowledge Graph, GraphRAG, Semantic Search, Large Language Models, Recommendation Systems, Community Intelligence.

3. INTRODUCTION

Community-driven technical organisations have become an important bridge between academic instruction and industry expectation. The Industry Skill Development and Innovation Cell (ISDIC) follows this model by establishing skill development centres inside partner colleges at no cost to the institution or the student, and by organising city-level community chapters around four domains: cybersecurity, artificial intelligence and machine learning, cloud and DevOps, and software development. Each domain follows a published year-long curriculum organised into three skill levels, with recurring workshops, laboratory sessions, competitions and a flagship event, supported by an advisory board of practising industry professionals.

The operational consequence of this model is a steadily growing and highly interconnected body of information. A single chapter generates records of members and their skill levels, curriculum modules and their prerequisites, tool stacks, event schedules and registrations, mentors, partner organisations and target outcomes. This information is inherently relational: a member is connected to a domain, a domain to a set of curriculum modules, a module to a set of tools, and an event to both the members who attended it and the skills it develops. Conventional database interfaces expose these records as isolated tables and require the user to know in advance where to look, while document-based content such as curriculum descriptions and event briefs remains entirely outside the reach of structured queries.

Recent advances in Generative AI, particularly Retrieval-Augmented Generation, allow a Large Language Model to answer questions using an external, authoritative corpus rather than relying solely on parametric memory. Standard RAG retrieves semantically similar text passages and is therefore effective for descriptive questions, but it performs poorly on queries that depend on relationships spanning multiple records, because the connecting information is never contained in a single passage. Knowledge Graphs address precisely this limitation by representing entities and their relationships explicitly, enabling multi-hop traversal and interpretable reasoning.

ISDIC communicated the requirement of an intelligent chatbot built over its existing community data. Taking that requirement as the problem context, this project proposes a broader Community Intelligence Platform in which a Knowledge Graph-enhanced RAG chatbot is the core module, supported by intelligent natural-language search with structured filtering, explainable recommendations, and AI-generated community insights for administrators. All four modules are deliberately built on a single shared retrieval and generation backbone, so that the system is realised as one integrated platform rather than four independent applications.

4. BACKGROUND

4.1 Problem Statement

To design and develop an integrated Community Intelligence Platform for ISDIC that employs a Knowledge Graph-enhanced Retrieval-Augmented Generation architecture to answer factual and relationship-based natural-language queries over the organisation's community data, and to extend the same retrieval and generation backbone to provide intelligent filtered search, explainable recommendations and generative community insights.

4.2 Motivation / Need of Project

The need for the proposed system arises from the following observations regarding the way community information is presently accessed and used:

- Community information is fragmented. Member profiles, chapter details, curriculum plans, event records and partner information are stored in different structured tables and unstructured documents, and no unified interface exists to query them together.
- Relationship-oriented questions cannot be answered directly. Queries of the form “which members in the AI and ML community have completed the intermediate level and have an interest in healthcare applications” require joining several records and are currently resolved manually.
- Coordination effort falls on chapter leads. Routine questions from students about eligibility, prerequisites, upcoming events and recommended learning paths consume time that would otherwise be spent on technical delivery.
- Discovery within the community is limited. Members are frequently unaware of peers, events or opportunities relevant to their skills and interests, which reduces the collaborative value of the community.
- Administrative visibility is reactive. The organisation lacks an automated mechanism to identify trending skills, recurring questions and knowledge gaps that should influence curriculum planning.
- A purely vector-based RAG chatbot would be insufficient. Semantic similarity retrieval cannot reliably answer multi-hop relational queries, which motivates the hybrid Knowledge Graph and vector retrieval design adopted in this project.

4.3 Objectives

The objectives of the proposed project are as follows:

- To design a data ingestion and preprocessing pipeline that consolidates ISDIC's structured community records and unstructured documents into a common processing format.
- To construct a domain-specific Knowledge Graph representing community entities such as members, skills, domains, chapters, institutions, curriculum modules, events, tools, mentors and opportunities, together with the relationships among them.
- To build a vector store of document and record embeddings supporting semantic retrieval over descriptive community content.

- To develop a hybrid retrieval mechanism that combines graph traversal, vector similarity search and structured filtering, and fuses the results into a single ranked context set.
- To implement a Generative AI chatbot that produces grounded, context-aware and source-attributed responses to factual as well as relationship-based natural-language queries.
- To extend the retrieval backbone into an intelligent search module supporting natural-language queries with structured filters on attributes such as skill, domain, chapter, location and skill level.
- To develop an explainable recommendation module that suggests relevant members, events, curriculum modules and opportunities, and generates a natural-language justification for each recommendation.
- To generate administrator-facing community insights, including trending topics, frequently asked questions and knowledge gaps, summarised in natural language by an LLM.
- To evaluate the system qualitatively on response groundedness, relational query coverage and retrieval relevance, and to deploy it as a single integrated web application.

4.4 Scope of the Project

Within Scope

- A working web-based platform comprising four integrated modules: RAG chatbot, intelligent search, recommendations and community insights.
- A Knowledge Graph constructed over ISDIC community entities, with an LLM-assisted entity and relation extraction pipeline for unstructured documents.
- A vector database of embeddings generated from curriculum content, event descriptions, member profile text and frequently asked questions.
- Hybrid retrieval combining graph traversal, semantic similarity and structured attribute filtering, with grounded response generation and source attribution.
- Role-based access separating member-facing modules from the administrator insights dashboard.
- Deployment of the complete system on a single cloud instance with documented APIs.

Outside Scope

- Training or fine-tuning a Large Language Model from scratch; the project uses a hosted or open-weight pre-trained model through an API.
- Replacement of ISDIC's existing operational systems for registration, attendance or administration; the platform reads from available data and does not act as a system of record.
- Real-time voice interaction, multilingual support and mobile application development, which are identified as possible future extensions.
- Automated decision-making on behalf of the organisation; all outputs are advisory and presented for human review.

5. LITERATURE SURVEY

The literature relevant to this project spans four connected areas: retrieval-augmented generation, dense semantic retrieval, the integration of knowledge graphs with large language models, and the use of large language models in recommendation. The following studies were reviewed to establish the technical foundation of the proposed system and to identify the gap it addresses.

Lewis et al. introduced Retrieval-Augmented Generation, coupling a parametric sequence-to-sequence model with a non-parametric dense retriever so that generation is conditioned on documents retrieved at inference time. Karpukhin et al. demonstrated that dense passage retrieval based on learned dual encoders substantially outperforms sparse lexical retrieval for open-domain question answering, establishing embedding-based retrieval as the standard component of RAG pipelines. Both approaches, however, retrieve passages independently and therefore treat the corpus as a flat collection of text.

Pan et al. surveyed the unification of large language models and knowledge graphs, presenting a roadmap of KG-enhanced LLMs, LLM-augmented KGs and synergised systems, and arguing that structured knowledge supplies the factual grounding and interpretability that black-box models lack. Edge et al. operationalised this direction in GraphRAG, extracting an entity-relation graph from a corpus, detecting communities within it and generating hierarchical summaries that are combined at query time, reporting improved performance over vector-based RAG on global, corpus-level questions. Zhao et al. reviewed recommender systems in the era of large language models, showing that LLMs can improve generalisation and provide natural-language reasoning for their suggestions.

Alongside the academic literature, several deployed platforms were examined to understand the practical landscape. Enterprise knowledge assistants such as Glean and Microsoft Copilot provide semantic search and question answering over organisational content; professional networks such as LinkedIn provide member and opportunity recommendations from a large behavioural graph; and community platforms such as Discord, Slack and Circle provide discussion and event management with keyword search. These systems demonstrate the commercial viability of the individual capabilities, but they are either proprietary and expensive, dependent on very large interaction histories, or restricted to a single capability, and none is designed for a resource-constrained, multi-chapter skill development community of the type ISDIC operates.

Summary of Literature Survey:

Sr. No.	Title of the Paper	Author(s)	Year	Publisher / Conference	Methodology	Conclusion / What Has Been Done	Research Gap
1	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks	P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal et al.	2020	Advances in Neural Information Processing Systems (NeurIPS), Vol. 33	Combines a pre-trained sequence-to-sequence generator with a dense vector retriever over a Wikipedia index; retrieved passages are used as latent context during generation, trained end-to-end.	Established the RAG paradigm and showed that conditioning generation on retrieved evidence improves factual accuracy on knowledge-intensive tasks compared with purely parametric models.	Retrieval is limited to independently scored text passages. Relationships that span multiple documents are not represented, so multi-hop relational queries remain unsupported.
2	Dense Passage Retrieval for Open-Domain Question Answering	V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, W. Yih	2020	Proceedings of EMNLP 2020, Association for Computational Linguistics	Dual-encoder architecture trained with contrastive learning to embed questions and passages into a shared space, with retrieval performed by maximum inner-product search.	Demonstrated that learned dense retrieval significantly outperforms sparse lexical baselines such as BM25, and established embedding-based retrieval as the default component of RAG systems.	Purely semantic matching cannot enforce structured constraints such as skill level, domain or chapter membership, and offers no mechanism for explicit relational reasoning.
3	Unifying Large Language Models and Knowledge Graphs: A Roadmap	S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, X. Wu	2024	IEEE Transactions on Knowledge and Data Engineering, Vol. 36, No. 7	Systematic survey proposing three frameworks: KG-enhanced LLMs, LLM-augmented KGs, and synergised LLM plus KG systems for bidirectional reasoning.	Consolidated the field and showed that structured knowledge supplies the factual grounding, interpretability and traceability that standalone LLMs lack.	Presents a conceptual roadmap rather than an implemented system. No end-to-end reference architecture is given for a domain-specific community application.

Sr. No.	Title of the Paper	Author(s)	Year	Publisher / Conference	Methodology	Conclusion / What Has Been Done	Research Gap
4	From Local to Global: A Graph RAG Approach to Query-Focused Summarization	D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, J. Larson	2024	arXiv preprint arXiv:2404.16130 (Microsoft Research)	Extracts an entity-relation graph from a text corpus using an LLM, detects communities in the graph, generates hierarchical community summaries and combines them through a map-reduce answering step.	Showed that graph-structured retrieval outperforms conventional vector RAG on global, corpus-level questions while reducing context tokens at higher summary levels.	Optimised for summarisation over document corpora rather than for queries against structured organisational records, and does not integrate structured attribute filtering or recommendation.
5	Recommender Systems in the Era of Large Language Models (LLMs)	Z. Zhao, W. Fan, J. Li, Y. Liu, X. Mei, Y. Wang, J. Tang, Q. Li et al.	2024	IEEE Transactions on Knowledge and Data Engineering, Vol. 36	Survey of LLM adaptation strategies for recommendation, covering prompting, in-context learning, instruction tuning and fine-tuning across pre-training and inference stages.	Established that LLMs improve generalisation to unseen scenarios and can generate natural-language explanations for their recommendations, addressing the opacity of conventional models.	Assumes large-scale user-item interaction histories that a growing community organisation does not possess, and does not address cold-start recommendation grounded in a knowledge graph.

Overall Research Gap:

The reviewed literature establishes each required capability in isolation. Vector-based RAG grounds generation in retrieved text but cannot follow relationships across records; dense retrieval provides strong semantic matching but no structured constraint satisfaction; knowledge graph and LLM integration supplies relational reasoning and interpretability but is presented largely as a conceptual roadmap or as a summarisation-oriented pipeline; and LLM-based recommendation offers explainability but presumes large interaction histories that a growing community does not have. Deployed commercial platforms combine some of these capabilities, but they are proprietary, costly and designed for large enterprises rather than for multi-chapter skill development communities.

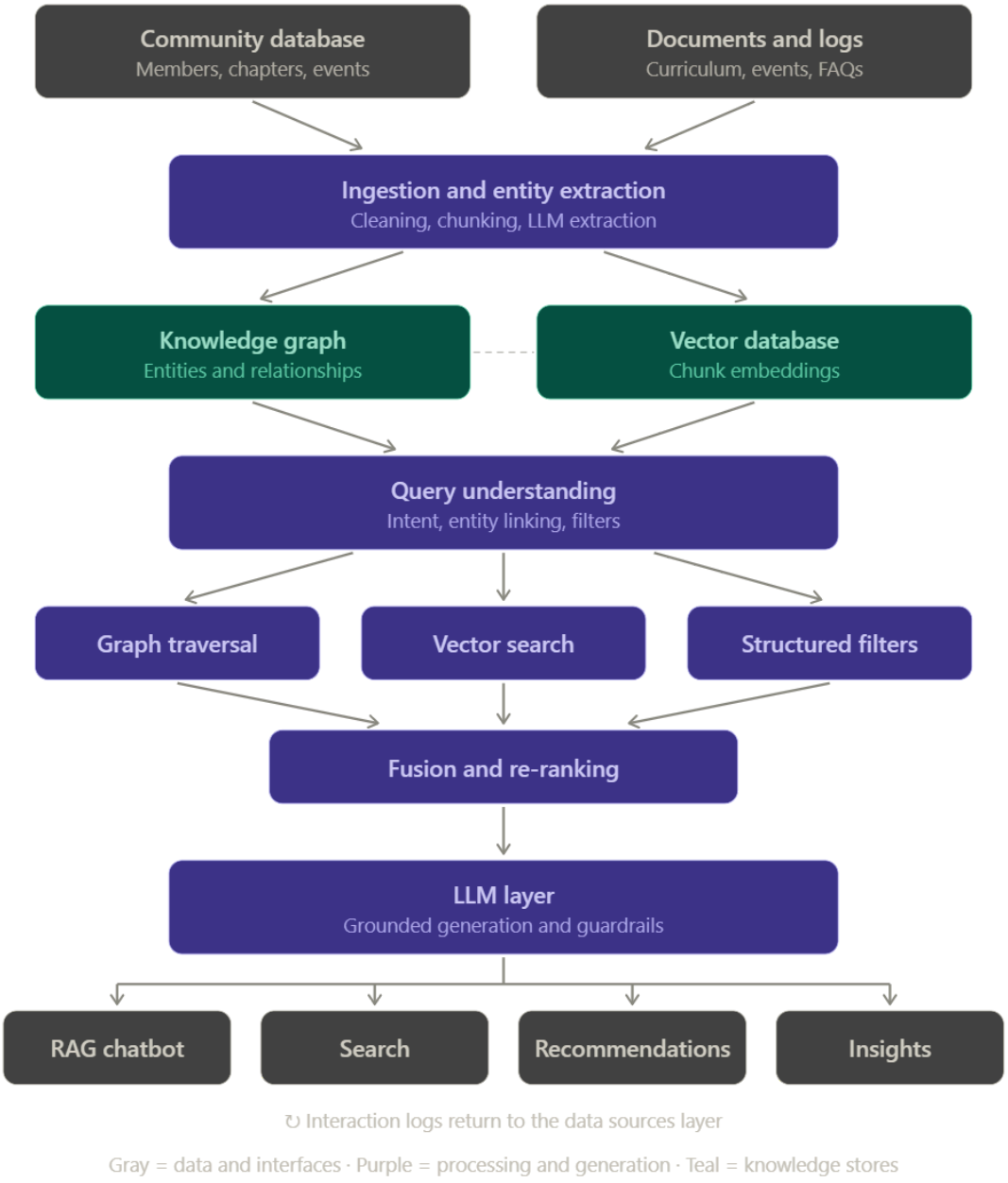
Consequently, there is no openly documented, resource-feasible architecture in which a knowledge graph and a vector store operate over the same community data to serve conversational question answering, natural-language search with structured filtering, explainable cold-start recommendation and generative administrative insight through a shared retrieval and generation backbone. The proposed Community Intelligence Platform addresses this gap by unifying these four capabilities over a single hybrid retrieval layer, using the knowledge graph both to answer relational queries and to justify recommendations, and using the LLM to convert retrieved evidence into grounded, attributable natural-language output.

6. METHODOLOGY

The proposed system is developed as a layered pipeline in which every module consumes the output of a common retrieval and generation backbone. The overall data flow is: community database, documents and other knowledge sources, followed by data processing and entity extraction, followed by parallel population of a knowledge graph and a vector database, followed by hybrid retrieval, semantic search and structured filtering, followed by the LLM layer, and finally the four application modules. This arrangement ensures the system behaves as one platform rather than as four independent features.

6.1 Block Diagram

The block diagram is organised into seven horizontal layers. Each layer is drawn as a labelled rectangle, and components within a layer are drawn as sub-blocks. Downward arrows connect consecutive layers; the specific connections are listed after the layer descriptions.



6.2 Hardware and Software Requirements

Hardware Requirements

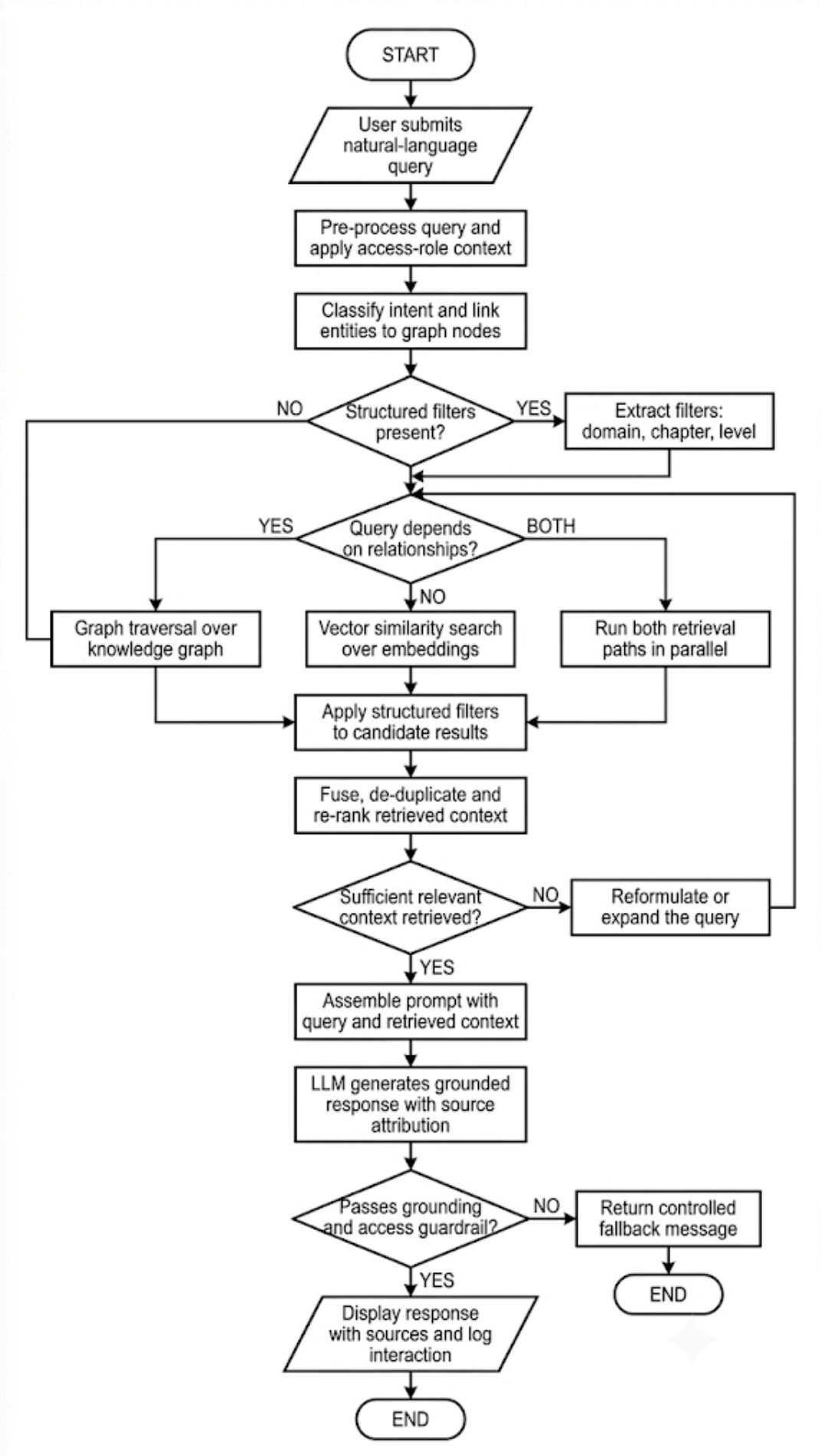
Component	Specification
Development System	Intel Core i5 / AMD Ryzen 5 processor or higher
RAM	16 GB (minimum 8 GB)
Storage	512 GB SSD
GPU (optional)	NVIDIA GPU with 6 GB VRAM, or Google Colab for embedding generation
Deployment Server	Cloud instance with 2 vCPU, 8 GB RAM, 50 GB storage

Component	Specification
Network	Stable broadband connection for LLM API access

Software Requirements

Category	Tools and Technologies
Operating System	Windows 11 / Ubuntu 22.04 LTS
Programming Language	Python 3.11
Backend Framework	FastAPI
Vector Database	PostgreSQL 16 with the pgvector extension
LLM Orchestration	LangChain / LangGraph
Embedding Model	Sentence-Transformers (all-MiniLM-L6-v2 or BGE-base)
Large Language Model	Google Gemini API, or an open-weight model such as Llama 3 served via API
Caching and Queue	Redis
Frontend	React (member interface and administrator dashboard)
Containerisation	Docker and Docker Compose
Version Control	Git and GitHub
Testing and Tools	Postman, Pytest

6.3 Flowchart


START
User submits natural-language query
Pre-process query and apply access-role context
Classify intent and link entities to graph nodes
Structured filters present?
NO
YES
Extract filters: domain, chapter, level
Query depends on relationships?
YES
BOTH
NO
Graph traversal over knowledge graph
Vector similarity search over embeddings
Run both retrieval paths in parallel
Apply structured filters to candidate results
Fuse, de-duplicate and re-rank retrieved context
Sufficient relevant context retrieved?
NO
Reformulate or expand the query
YES
Assemble prompt with query and retrieved context
LLM generates grounded response with source attribution
Passes grounding and access guardrail?
NO
Return controlled fallback message
END
YES
Display response with sources and log interaction
END

6.4 Working Principle

The working of the system is described below in terms of the pipeline stages and the four application modules.

A. Knowledge Base Construction

Community data is ingested from ISDIC's existing records and documents. Structured records are mapped directly onto the graph schema, so that a member record becomes a Member node connected by MEMBER_OF to a Chapter node and by BELONGS_TO_DOMAIN to a Domain node. Unstructured content is chunked and passed to an LLM with a schema-constrained extraction prompt that returns entity-relation triples, which are validated against the permitted entity and relationship types before insertion into Neo4j. The same chunks are embedded using a sentence-transformer model and stored in PostgreSQL with pgvector, each embedding carrying metadata that includes the identifiers of the entities it mentions. This shared identifier is what makes hybrid retrieval possible: a node reached through graph traversal can be expanded with its supporting text, and a text chunk retrieved semantically can be expanded with its graph neighbourhood.

B. Query Understanding and Hybrid Retrieval

An incoming query is classified by intent and scanned for entity mentions and structured constraints. Descriptive queries such as “what does the cloud and DevOps track cover” are routed primarily to vector retrieval. Relational queries such as “which members have completed the intermediate AI and ML level and are interested in healthcare” are routed to graph traversal, where a parameterised Cypher query walks the relevant relationships. Mixed queries invoke both paths concurrently. Structured filters are applied as constraints on the candidate set rather than as post-processing, which preserves recall. The candidate results from all paths are fused, de-duplicated by entity identifier and re-ranked by combined semantic and structural relevance.

C. Module 1 – Knowledge Graph-Enhanced RAG Chatbot

The core module assembles the fused context into a structured prompt instructing the LLM to answer only from the supplied evidence and to cite its sources. Because relational evidence arrives as explicit graph paths, the model can state not merely that two members are relevant but how they are connected, and a guardrail stage rejects any response that cannot be traced to retrieved evidence. If retrieval is insufficient, the system states this rather than generating an unsupported answer.

D. Module 2 – Intelligent Community Search with Filters

The search module reuses the same retrieval layer but returns a ranked list of entities instead of a conversational answer. The LLM interprets the natural-language query and converts it into a combination of semantic intent and structured constraints, so that a request such as “beginner-friendly cybersecurity sessions in the Pune chapter” is resolved as a semantic match on session content, constrained by chapter and skill level. Users may additionally apply filters directly through the interface, and results are returned with a brief generated summary of why each entity matched.

E. Module 3 – AI-Powered Recommendations

Recommendations are generated primarily from graph structure rather than from interaction history, which allows the module to function despite a limited behavioural record. Candidate entities are identified through relationship patterns such as shared domains, overlapping skills, complementary skill levels, common event attendance and curriculum prerequisites, and are then re-ranked using embedding similarity between profile text. For each recommendation, the traversed graph path is supplied to the LLM, which produces a short natural-language justification, for example that a member is suggested because both work in generative AI and have contributed to the same open-source sprint. The explanation therefore reflects an actual path in the graph rather than a post-hoc rationalisation.

F. Module 4 – Generative Community Insights

The insights module operates over aggregated interaction logs and graph statistics. Query logs are clustered by embedding similarity to identify recurring themes and frequently asked questions; graph queries surface the distribution of skills, domains and event participation across chapters; and comparison of demanded topics against covered curriculum modules identifies knowledge gaps. These aggregates are passed to the LLM, which produces a narrative summary explaining what the pattern is, why it may have arisen and what action it suggests. The emphasis is on generated explanation rather than on numerical dashboards alone, and access is restricted to administrator roles.

G. Integration

All four modules are served by a single FastAPI backend and share the same ingestion pipeline, knowledge stores, retrieval layer and LLM layer. Adding a new data source therefore improves every module simultaneously, and the interaction logs generated by the member-facing modules become the input to the insights module, closing the loop within a single integrated platform.

7. EXPECTED CONCLUSIONS

On completion, the project is expected to deliver the following outcomes:

- A deployed Community Intelligence Platform for ISDIC in which a Knowledge Graph-enhanced RAG chatbot, intelligent filtered search, explainable recommendations and generative community insights operate over a shared retrieval and generation backbone.
- A populated domain-specific knowledge graph representing ISDIC community entities and their relationships, constructed through a repeatable ingestion and LLM-assisted extraction pipeline that can be re-executed as the community grows.
- Demonstrated capability to answer relationship-based queries that vector-only retrieval cannot resolve, with the graph path supplied as visible justification for the answer.
- Grounded and source-attributed responses, with an explicit refusal behaviour when retrieved evidence is insufficient, reducing the risk of unsupported generation.
- A recommendation mechanism that functions under cold-start conditions by relying on graph structure rather than large interaction histories, and that accompanies each suggestion with a natural-language explanation derived from an actual relationship path.
- An administrator-facing insights module that converts aggregated community activity into narrative summaries of trending topics, recurring questions and curriculum knowledge gaps.
- A reduction in the manual coordination effort presently borne by chapter leads, and improved discovery of relevant peers, events and opportunities by community members.
- A modular and documented architecture that can be extended to additional ISDIC chapters, or adapted to other community organisations with comparable data structures, without redesign of the retrieval backbone.

The project is also expected to contribute a practical reference architecture demonstrating that a hybrid Knowledge Graph and vector retrieval design can support multiple Generative AI capabilities over a single knowledge base within the resource constraints of a student-developed system, an approach that the reviewed literature discusses conceptually but does not document as an integrated, deployed platform.

8. REFERENCES

[1]P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020.

[2]V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen and W. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics, pp. 6769–6781, 2020.

[3]S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang and X. Wu, "Unifying Large Language Models and Knowledge Graphs: A Roadmap," IEEE Transactions on Knowledge and Data Engineering, vol. 36, no. 7, pp. 3580–3599, July 2024.

[4]D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt and J. Larson, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," arXiv preprint arXiv:2404.16130, 2024.

[5]Z. Zhao, W. Fan, J. Li, Y. Liu, X. Mei, Y. Wang, Z. Wen, F. Wang, X. Zhao, J. Tang and Q. Li, "Recommender Systems in the Era of Large Language Models (LLMs)," IEEE Transactions on Knowledge and Data Engineering, vol. 36, pp. 6889–6907, 2024.

[6]Industry Skill Development and Innovation Cell (ISDIC), Official Website. [Online]. Available: https://isdic.in

[7]Industry Skill Development and Innovation Cell (ISDIC), Community Curriculum. [Online]. Available: https://isdic.in/curriculum

[8]pgvector, Open-Source Vector Similarity Search for PostgreSQL. [Online]. Available: https://github.com/pgvector/pgvector

[9]LangChain, LangChain and LangGraph Documentation. [Online]. Available: https://python.langchain.com/docs